

Total No. of Questions : 10]

SEAT No. :

P1670

[Total No. of Pages : 3

[5460]-197

**T.E. (Information Technology)
SYSTEMS PROGRAMMING
(2012 Pattern) (Semester-II)**

Time : 2½ Hours]

[Max. Marks : 70

Instructions to the candidates:

- 1) *Neat diagrams must be drawn wherever necessary.*
- 2) *Figures to the right indicate full marks.*
- 3) *Your answers will be valued as whole.*
- 4) *Assume suitable data, if necessary.*

Q1) a) Explain how a single pass assembler works for the following example

```
START 300
READ A
LOOP MOVER AREG,A
      SUB AREG, ='1'
      BC GT, LOOP
      STOP
A    DS 1
      END
```

Explain the terms forward reference and backpatching with respect to the example given. [6]

b) Compare Compile and Go Loader with Absolute Loader. [4]

OR

Q2) a) Convert the given Regular expression to its equivalent DFA: [6]

$((a+ba)^*(b+c))+b^*$

b) With examples explain the functions of START, END, LTORG, ORIGIN and EQU. [4]

Q3) a) Explain first three phases of a compiler for the given statement: [6]

$(X-Y)*Y^ZP/A$

b) Give the flowchart for pass II of DLL scheme. [4]

P.T.O.

OR

- Q4)** a) Give the various data structures in the design of pass-1 and pass-2 of a Two-pass direct linking loader for the given example. [6]

	Rel. Addr.
PGA START	0
ENTRY PGA1	10
EXTRN PGB	20
DC A (PGA), A(PGB+4)	30
PGA1 DC A (PGA1-PGA)	34
END	
PGB START	0
ENTRY PGB1	5
EXTRN PGA	10
PGB1 DC A (PGB1)	14
PGB2 DC A (PGB+4), A(PGB1-PGB)	18
PGB3 DC A (PGB-PGA-16)	22
END	

- b) Compare single pass and two pass assembler. [4]

- Q5)** a) With a neat figure explain classification of parsers. [8]

- b) Find FIRST and FOLLOW of non terminals in the given grammar: [4]

$S \rightarrow (L)/a$

$L \rightarrow L, S/S$

- c) Remove ambiguity and make the give grammar left recursion free [6]

$S \rightarrow S+S/SS/(S)/S*S/a$

OR

- Q6)** a) Compare SLR and CLR parsing methods. [4]

- b) Design LALR parser for the given grammar. Also show the moves by the parser for input string "ab". [8]

$S \rightarrow AA$

$A \rightarrow aA/b$

- c) Using the table given, design operator precedence parser for the given expression $\text{id} * \text{id} + \text{id}$ [6]

	+	-	*	/	^	id	(	)	S
+	>	>	<	<	<	<	<	>	>
-	>	>	<	<	<	<	<	>	>
*	>	>	>	>	<	<	<	>	>
/	>	>	>	>	<	<	<	>	>
^	>	>	>	>	<	<	<	>	>
id	>	>	>	>	>			>	>
(	<	<	<	<	<	<	<		
)	>	>	>	>	>			>	>
S	<	<	<	<	<	<	<		

- Q7)** a) Define and explain annotated parse tree for the given grammar. [8]

$D \rightarrow TL$

$T \rightarrow \text{int/float}$

$L \rightarrow L, \text{id/id}$

Annotate the tree for float id1, id2, id3

- b) Explain the need of Intermediate code generation in compiler.
Generate quadruple and triple for the given expression: $a = b + c * d$. [8]

OR

- Q8)** a) Draw the Dependency graph of Type expression of Q.7.a) and list down the synthesized and inherited attributes with definition. [8]

- b) Write the method of generating intermediate code for the expression $(a < b)$ and $(c > d)$ [8]

- Q9)** a) Discuss with suitable example machine dependent code optimization. [8]

- b) Explain different intermediate code generation techniques for the given expression. [8]

$a + a * (b - c) + (b - c) * d$

OR

- Q10)** a) Obtain the TAC for the following code [8]

for ($i = 1$; $i \leq 10$; $i++$)

$X[i][2*j-1] = Y[i][2*j-1]$

- b) Discuss code generation issues. [4]
c) Write a short note on activation record. [4]

